

Ejercicios evaluables UA3

Esta actividad aborda los criterios de evaluación del RA.

Instrucciones

● Todas las actividades que se realicen deben entregarse mediante un enlace al repositorio de Github y un documento (si se pide alguna evidencia o se plantean preguntas)

DAW_ OPT_UA3_ApellidosNombre.pdf

- Las entregas deberán realizarse en la tarea correspondiente en el formato especificado antes de la hora de finalización. Cualquier tarea que no cumpla estos requisitos tendrá una calificación de 0.
- Se pedirá que defiendas tu práctica. Si no puedes justificar tu trabajo o responder las preguntas que se te plantean, la calificación de la tarea será de 0.

Sobre el uso de IA y fuentes externas

Puedes utilizar herramientas como buscadores o inteligencia artificial como apoyo, pero no como sustituto de tu trabajo. Las respuestas deben estar redactadas por ti, con tus propias ideas y razonamientos.

Se valorará especialmente el análisis personal y la justificación razonada.

Para Python/Django

Ejercicio 1

En DWES Semana 1 se ha implementado la creación de entradas de biblioteca (en la ruta **POST /api/library/entries**). En este ejercicio debes escribir una batería de tests automáticos que garantice que esta ruta cumple exactamente el comportamiento definido.

Debes cubrir, al menos, los siguientes casos:

- ☐ Caso válido
 - Envío de un JSON correcto.
 - Comprobación de:
 - código 201
 - estructura exacta del JSON de respuesta
- ☐ Errores de validación (400 validation_error)
 - JSON vacío.
 - Falta de un campo obligatorio.
 - Tipo incorrecto en hours_played.
 - Valor negativo en hours_played.
 - Valor de status fuera de los permitidos.
- ☐ Entrada duplicada
 - Creación correcta de una entrada.
 - Segundo intento con el mismo external_game_id.
 - Comprobación de 400 duplicate_entry.

En todos los errores debes comprobar:

- ☐ código HTTP,
- ☐ que la respuesta es JSON,
- ☐ que el contrato de error se cumple exactamente (es decir, que haya una clave error con un valor adecuado, el message sea el correcto...).

Ejercicio 2

Se ha implementado una ruta accesible mediante **GET a /api/library/entries/**, que devuelve el listado completo de la biblioteca.

Escribe tests que comprueben:

- ☐ el comportamiento cuando la biblioteca está vacía,
- ☐ el comportamiento cuando existen varias entradas,
- ☐ el formato correcto de los datos devueltos.

Ejercicio 3

La Semana 1 de DWES incluye una ruta accesible mediante **GET a /api/library/entries/{id}/**, que devuelve el detalle de una entrada concreta.

Escribe tests que comprueben:

- ☐ el acceso correcto a una entrada existente,
- ☐ la respuesta adecuada cuando el id no existe.

Ejercicio 4

La Semana 1 de DWES incluye una ruta accesible mediante **PATCH a /api/library/entries/{id}/**, que permite modificar parcialmente una entrada existente.

Escribe tests automáticos que comprueben:

- ☐ una actualización parcial correcta,
- ☐ varios casos de error de validación,
- ☐ la respuesta cuando se intenta actualizar una entrada inexistente.

Para Java

Ejercicio 1

Crea el archivo: `src/test/java/PriorityRulesTest.java`

Debe incluir exactamente 5 tests y cubrir todos los escenarios:

1. `parsePriority_ok_minimoYmaximo`
 - Probar "1" y "5".
 - Comprobar que devuelve 1 y 5.
2. `parsePriority_falla_siNull`
 - Probar null.
 - Debe lanzar `IllegalArgumentException`.
3. `parsePriority_falla_siNoNumero`
 - Probar "abc".
 - Debe lanzar `IllegalArgumentException`.
4. `parsePriority_falla_fueraDeRango`
 - Probar "0" y "6".
 - Debe lanzar `IllegalArgumentException`.
5. `validatePriority_okYerror`
 - `validatePriority(3)` no lanza.
 - `validatePriority(0)` lanza y se comprueba el mensaje.

Ejercicio 2

Crea el archivo: `src/test/java/TagRulesTest.java`

Debe incluir exactamente 6 tests:

1. `normalizeTag_ok_minimo`
 - Entrada: "casa" → salida: "casa".
2. `normalizeTag_ok_recortaYMinusculas`
 - Entrada: " CaSa " → salida: "casa".
3. `normalizeTag_fallaSiNull`
 - Entrada: null → excepción + mensaje.
4. `normalizeTag_fallaSiVacio`
 - Entrada: " " → excepción + mensaje.
5. `normalizeTag_fallaSiDemasiadoLargo`
 - Entrada: string de longitud 31 → excepción + mensaje.
6. `normalizeTag_fallaSiCaracteresInvalidos`
 - Entrada: "hola!" → excepción + mensaje.

Ejercicio 3

Crea el archivo: `src/test/java/TaskStatsServiceTest.java`

Importante: Aunque el servicio trabaje con listas en el proyecto, en este ejercicio:

- ☐ No se permiten listas/arrays en el test.

Debe incluir exactamente 2 tests:

1. `doneRatio_devuelveCero_siNull`
 - ☐ Llamar a `doneRatio(null)` y comprobar que devuelve 0.0.
2. `doneCount_devuelveCero_siNull`
 - ☐ Llamar a `doneCount(null)` y comprobar que devuelve 0.

Rúbrica de evaluación de la tarea

Criterio de evaluación	Nivel alto (9-10)	Nivel medio (5-9)	Nivel Bajo (1-5)	No logrado (0)